

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

PROJECT REPORT

ScanSight

Automatic Document Scanner & Quality Analyzer

Submitted as part of the course requirements for
Computer Vision

Name	Anunay Chhapre
Registration Number	24BAI10425
Programme	B.Tech Computer Science and Engineering (AI & ML)
Course	Computer Vision
Project Area	Classical Digital Image Processing & Document Analysis
Repository	github.com/Anunay1601/Computer-Vision-24BAI10425

2026

Table of Contents

1. Abstract
2. Introduction
3. Project Objectives
4. Related Work / References Consulted
5. System Design and Methodology
6. Implementation Details
7. Results and Generated Outputs
8. Testing
9. Limitations
10. Applications
11. Conclusion and Future Scope
12. References

1. Abstract

ScanSight is a command-line computer vision system that converts a photograph of a document into a clean, flattened, scanner-like output and reports objective measures of its visual quality. Rather than relying on a trained deep-learning detector, the system is built entirely on classical digital image processing techniques — grayscale conversion, Gaussian smoothing, Canny edge detection, contour extraction, polygon approximation and a four-point perspective transform — followed by an enhancement stage (denoising, CLAHE contrast enhancement and adaptive thresholding) and a quality-analysis stage that estimates brightness, contrast, sharpness and skew. The pipeline is implemented in Python using OpenCV, NumPy and Pillow, is fully testable through Python's *unittest* framework, and produces three deliverables per run: the enhanced scanned image, a visual processing report, and a machine-readable JSON quality report. The project demonstrates how a practical, real-world document-scanning problem can be solved end-to-end using only classical image-processing theory, without depending on annotated training data or GPU resources.

2. Introduction

Photographing paper documents with a phone camera is now the most common way people digitize receipts, forms, notes and printed pages. Unlike a flatbed scanner, however, a handheld photograph rarely captures the document as a flat, evenly lit rectangle: it is affected by perspective distortion, uneven background, variable lighting and camera noise. **ScanSight** addresses this gap by automatically detecting the boundary of a document within a photograph, correcting the perspective distortion so the page appears as if scanned from directly above, enhancing the resulting image for readability, and then quantifying how good the final scan actually is.

The project deliberately restricts itself to **classical digital image processing and document-analysis techniques** rather than deep-learning-based document detectors. This choice keeps the entire pipeline interpretable: every stage of the process (edge map, contour, quadrilateral, transform matrix, enhanced image) can be inspected and reasoned about directly, which is well suited to a course focused on the theory and mechanics of computer vision rather than on training neural networks. The complete system is exposed as a command-line tool, making it straightforward to install, run and evaluate without a graphical interface.

3. Project Objectives

- Apply classical computer vision techniques to a practical document-processing problem.
- Automatically identify document boundaries within a photographed image.
- Correct perspective distortion in photographed documents using a four-point transform.
- Improve document readability through denoising and contrast/threshold enhancement.
- Measure document image quality using objective, reproducible metrics.
- Generate structured, machine-readable processing results (JSON) alongside visual outputs.
- Provide a complete, terminal-based execution flow suitable for evaluation without a GUI.
- Demonstrate a practical, end-to-end application of digital image processing theory.

4. Related Work / References Consulted

The design of ScanSight draws on well-established results in classical image processing and computer vision. Edge-based document-boundary detection and perspective correction are standard techniques described in general image-processing and computer-vision references, and the OpenCV library's own documentation provides the specific building blocks — edge detection, contour finding and geometric transforms — used to implement the pipeline. These sources are listed in full in the References section.

5. System Design and Methodology

5.1 Computer Vision Pipeline

The system processes an input photograph through a sequential pipeline of classical image processing stages, illustrated below.

```
Input Document Image
|
v
Image Loading -> Grayscale Conversion -> Gaussian Blur
|
v
Canny Edge Detection -> Dilation -> Contour Detection
|
v
Polygon Approximation -> Document Boundary Found
|
v
Four-Point Perspective Transform -> Scanned Document
|
v
Enhancement: Denoising + CLAHE Contrast + Adaptive Thresholding
|
v
Quality Analysis: Brightness, Contrast, Sharpness, Skew
|
v
Output Image + Visual Report + JSON Quality Report
```

Figure 1. End-to-end ScanSight processing pipeline.

5.2 Stage-wise Methodology

5.2.1 Image Preprocessing

The input image is converted to grayscale and smoothed with a Gaussian blur to suppress noise before edge detection.

5.2.2 Edge Detection

Canny edge detection identifies the significant intensity boundaries in the image, followed by dilation to close small gaps in the edge map.

5.2.3 Contour Detection

Contours are extracted from the edge map as candidate boundaries for the document region.

5.2.4 Polygon Approximation

Each candidate contour is approximated as a polygon to test whether it reduces to a four-sided shape consistent with a document edge.

5.2.5 Perspective Transformation

The four detected corner points are used to compute and apply a perspective transform, producing a top-down, rectangular view of the document.

5.2.6 Image Enhancement

The flattened document is denoised, contrast-enhanced with CLAHE (Contrast-Limited Adaptive Histogram Equalization), and binarized with adaptive thresholding to improve readability.

5.2.7 Quality Analysis

The final image is scored on brightness, contrast, sharpness and skew to give an objective measure of scan quality.

5.2.8 Report Generation

The scanned image, an annotated visual report, and a JSON file of quality metrics are written to the outputs directory.

6. Implementation Details

6.1 Technology Stack

Component	Technology
Programming Language	Python 3.10+
Core CV Library	OpenCV
Numerical Computing	NumPy
Image I/O	Pillow
Testing Framework	Python unittest
Documentation Diagrams	Mermaid

6.2 Project Structure

```
Computer-Vision-24BAI10425/  
|-- docs/  
| |-- project_report.md  
| `-- diagrams/  
|-- outputs/  
| |-- scanned_document.png  
| |-- visual_report.png  
| `-- quality_report.json  
|-- samples/  
|-- scripts/  
| `-- generate_sample.py
```

```
| -- src/scansight/  
| | -- cli.py  
| | -- config.py  
| | -- io_utils.py  
| | -- preprocessing.py  
| | -- scanner.py  
| | -- quality.py  
| | -- report.py  
| -- tests/  
| | -- test_scansight.py  
| -- main.py  
| -- pyproject.toml  
| -- requirements.txt  
|-- README.md
```

6.3 Installation and Execution

The project is installed and run entirely from the command line, which makes it suitable for terminal-based evaluation without a graphical development environment.

1. Clone the repository and move into the project directory.
2. Create and activate a Python virtual environment (Python 3.10 or newer).
3. Install dependencies with `pip install -r requirements.txt` and install the package in editable mode with `pip install -e .`
4. Run the demo pipeline with `python main.py --cli`, or process a specific image with `python main.py --cli --input path/to/document.jpg` (supported formats: JPG, JPEG, PNG, BMP).

7. Results and Generated Outputs

After a successful run, ScanSight writes its results to the `outputs/` directory:

- **scanned_document.png** — the perspective-corrected and enhanced document image.
- **visual_report.png** — a visual representation of the processing stages and quality analysis.
- **quality_report.json** — machine-readable quality measurements, including brightness, contrast, sharpness and skew; exact values depend on the input image.

8. Testing

The project includes automated unit tests covering the core computer vision functionality, located in `tests/test_scansight.py`. All tests can be run from the project root with `python -m unittest discover -s tests`. A successful test run indicates that the core preprocessing, scanning and quality-analysis components are functioning correctly.

9. Limitations

Because ScanSight relies on classical, rule-based image processing rather than a trained deep-learning detector, its performance can degrade under certain conditions:

- The document boundary is not clearly visible against the background.

- The image contains heavy background clutter.
- The document is severely blurred.
- Lighting in the scene is extremely uneven.
- Large portions of the document are occluded.
- The document has an irregular or damaged boundary.
- The input image has very low resolution.

10. Applications

- Digital document scanning and digitization.
- Pre-processing for Optical Character Recognition (OCR) pipelines.
- Automated document quality assessment.
- Mobile document-scanning applications.
- Digital archiving of physical records.
- Automated, terminal-based document-processing systems.

11. Conclusion and Future Scope

ScanSight demonstrates that a full document-scanning workflow — boundary detection, perspective correction, enhancement and objective quality assessment — can be built end-to-end using classical digital image processing techniques, without a trained deep-learning model. The command-line design, automated test suite and structured JSON/visual reporting make the system reproducible and easy to evaluate. Future work could extend the project by incorporating a learned document-corner detector to improve robustness under cluttered backgrounds and occlusion, adding OCR as a downstream stage, and building a lightweight mobile or web front-end around the existing processing pipeline.

12. References

- 1 OpenCV Documentation — Image Processing, Edge Detection, Contours and Perspective Transformation.
- 2 Gonzalez, R. C. and Woods, R. E., *Digital Image Processing*.
- 3 Szeliski, R., *Computer Vision: Algorithms and Applications*.